Probabilistic Machine Learning

Block 1 – Introduction & Probabilistic Regression

Andrés Masegosa

Aalborg University

2026

Outline

1. Introduction to Probabilistic Machine Learning
2. Probabilistic Linear Regression

Machine Learning Is Everywhere... But Can We Trust It?

ML models make predictions that affect **real decisions**:

A diagnostic model says
"*benign tumour*"

A credit model says
"*approve the loan*"

A self-driving car says
"*no obstacle ahead*"

A weather model says
"*no hurricane risk*"

Machine Learning Is Everywhere... But Can We Trust It?

ML models make predictions that affect **real decisions**:

A diagnostic model says
"*benign tumour*"

A credit model says
"*approve the loan*"

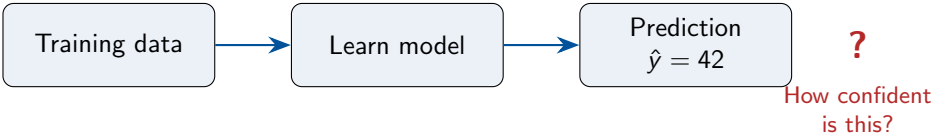
A self-driving car says
"*no obstacle ahead*"

A weather model says
"*no hurricane risk*"

In every case, a single number is not enough.

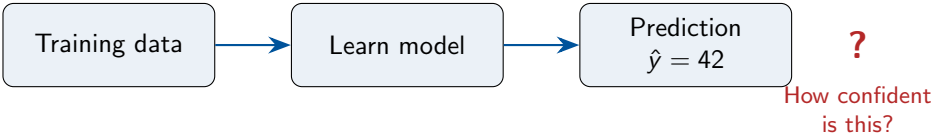
What's Missing in Standard ML?

The **classical pipeline**:



What's Missing in Standard ML?

The **classical pipeline**:



Standard ML models typically **cannot**:

- Express **confidence** — are all predictions equally reliable?
- Say **"I don't know"** — what happens in regions with no data?
- Quantify **noise** — how much variability is inherent in the problem?

Standard ML gives us **answers**, but not how much to **trust** them.

The Probabilistic Perspective — Intuition

Classical approach:

"What will the temperature be tomorrow?"

$\longrightarrow$ **22°C**

A single number.
Take it or leave it.

The Probabilistic Perspective — Intuition

Classical approach:

"What will the temperature be tomorrow?"

$\longrightarrow$ **22°C**

A single number.
Take it or leave it.

vs.

Probabilistic approach:

"What will the temperature be tomorrow?"

$\longrightarrow$ **Most likely 22°C**,
probably between 20–24°C.

A **distribution**
over possible answers.

The Probabilistic Perspective — Intuition

Classical approach:

"What will the temperature be tomorrow?"

$\longrightarrow$ **22°C**

A single number.
Take it or leave it.

vs.

Probabilistic approach:

"What will the temperature be tomorrow?"

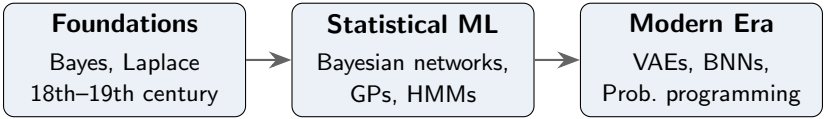
$\longrightarrow$ **Most likely 22°C**,
probably between 20–24°C.

A **distribution**
over possible answers.

Core idea: Instead of predicting a single value,
predict a **probability distribution** over all possible values.

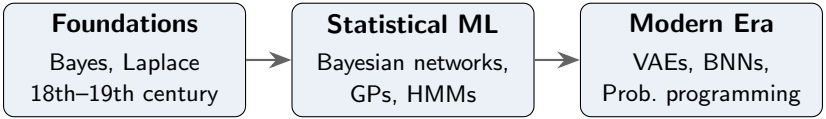
Probabilistic ML in Context

Probabilistic thinking in ML is **not new** — but modern tools have made it practical:



Probabilistic ML in Context

Probabilistic thinking in ML is **not new** — but modern tools have made it practical:



Today: languages like **Pyro**, Stan, and NumPyro let us:

- Write models in a few lines of Python.
- Perform inference **automatically** using gradient-based methods.
- Scale to large datasets and deep neural networks.

This course: learn the **ideas** and the **tools** to put them into practice.

What Does a ML Model Return?

When we train a model, we typically get a **point prediction**:

$$f(x) = \hat{y}$$

"The price of this house is €250,000."

What Does a ML Model Return?

When we train a model, we typically get a **point prediction**:

$$f(x) = \hat{y}$$

"The price of this house is €250,000."

But how much should we **trust** this prediction?

- Is it equally reliable for all inputs?
- What if the model has never seen similar data?
- Would we make the same decision regardless of confidence?

From Functions to Distributions

Classical ML: learn a function

$$x \longrightarrow \hat{y} = f(x)$$

From Functions to Distributions

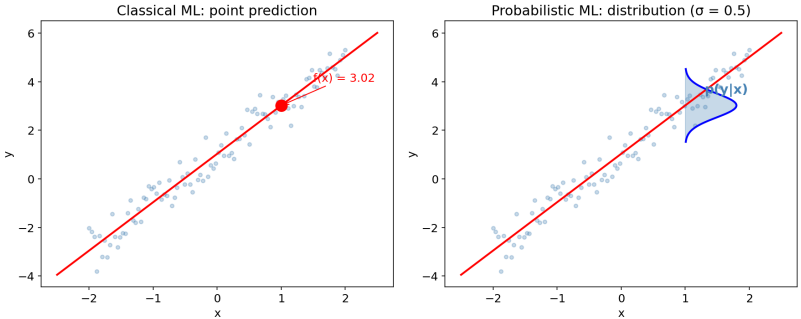
Classical ML: learn a function

$$x \longrightarrow \hat{y} = f(x)$$

Probabilistic ML: learn a *conditional distribution*

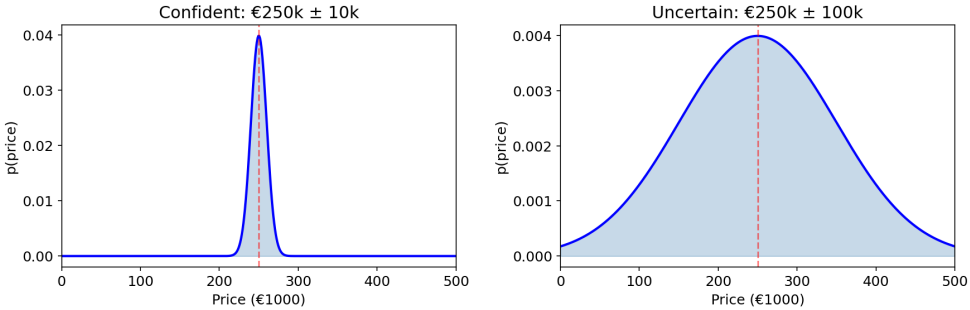
$$x \longrightarrow p(y \mid x)$$

Not a single answer, but a **distribution over possible answers**.



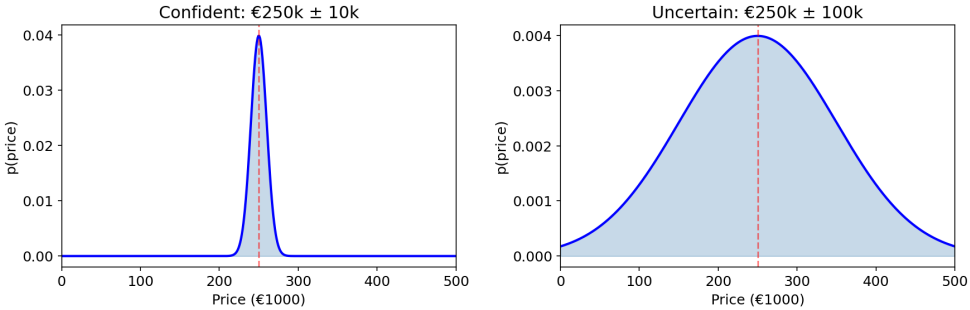
Why Uncertainty Matters

Two predictions with the **same mean** but very different uncertainty:



Why Uncertainty Matters

Two predictions with the **same mean** but very different uncertainty:



- **Left:** "€250k ± 10k" — confident, act on it.
- **Right:** "€250k ± 100k" — uncertain, be cautious.

Same prediction, very different decisions.

When Does Uncertainty Matter Most?

Noisy data
Observations are inherently imprecise.

Sparse regions
Few training points $\Rightarrow$ high uncertainty.

Ambiguous inputs
Multiple valid outputs for the same input.

Risk-sensitive decisions
Medicine, autonomous driving, finance...
The *cost of being wrong* varies.

Probabilistic ML lets us distinguish confident from uncertain predictions.

A Unifying Framework

Many classical algorithms are **special cases** of probabilistic models:

Classical method	**Probabilistic model**	**Distribution**
Linear regression	Gaussian linear model	$\mathcal{N}(\mathbf{w}^\top \mathbf{x} + b,\ \sigma^2)$
Logistic regression	Bernoulli model	$\mathrm{Bernoulli}(\sigma(\mathbf{w}^\top \mathbf{x} + b))$
k-means	Gaussian mixture model	$\sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$
PCA	Probabilistic PCA	Linear latent variable model

A Unifying Framework

Many classical algorithms are **special cases** of probabilistic models:

Classical method	**Probabilistic model**	**Distribution**
Linear regression	Gaussian linear model	$\mathcal{N}(\mathbf{w}^\top \mathbf{x} + b,\ \sigma^2)$
Logistic regression	Bernoulli model	$\mathrm{Bernoulli}(\sigma(\mathbf{w}^\top \mathbf{x} + b))$
k-means	Gaussian mixture model	$\sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$
PCA	Probabilistic PCA	Linear latent variable model

These are not isolated algorithms — they are **instances of the same framework**.

Decoupling Modeling and Inference

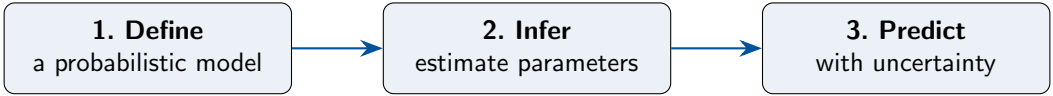
In classical deep learning these are often **tightly coupled**.

In probabilistic ML they are **separate concerns**:

1. Define a probabilistic model of the data.
2. Choose an inference method (MLE, variational inference, . . .).

PML precisely supports this separation.

The Probabilistic ML Recipe

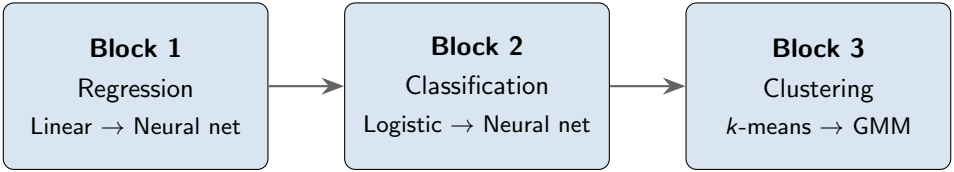


We will follow this recipe **three times today**:

1. **Block 1:** Regression
2. **Block 2:** Classification
3. **Block 3:** Clustering (unsupervised)

Same framework, same tool (Pyro), increasing complexity.

Today's Roadmap

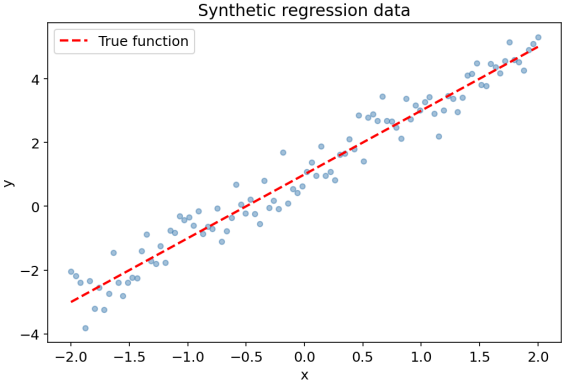


Each block:

1. Conceptual presentation
2. Pyro notebook (simple model)
3. Extension to more expressive models
4. Pyro notebook (extended model)

Regression: The Setup

Given pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, we want to predict y from $\mathbf{x}$.



How should we model the relationship between $\mathbf{x}$ and y?

The Standard Approach

Fit a linear function minimizing **Mean Squared Error**:

$$\hat{\mathbf{w}}, \hat{b} \;=\; \arg\min_{\mathbf{w},b} \frac{1}{N} \sum_{i=1}^{N} \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2$$

The Standard Approach

Fit a linear function minimizing **Mean Squared Error**:

$$\hat{\mathbf{w}}, \hat{b} \;=\; \arg\min_{\mathbf{w},b} \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \mathbf{w}^\top \mathbf{x}_i - b\right)^2$$

This works, but raises questions:

- Where does MSE come from? Why this loss and not another?
- How confident are we in the predictions?
- Can we quantify the noise in the data?

A probabilistic perspective answers all three.

A Probabilistic Model for Regression

Assumption: observations are noisy versions of a linear function.

$$y = f_\theta(\mathbf{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

A Probabilistic Model for Regression

Assumption: observations are noisy versions of a linear function.

$$y = f_\theta(\mathbf{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

With $f_\theta(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$, this means:

$$\boxed{y \mid \mathbf{x} \;\sim\; \mathcal{N}\big(\mathbf{w}^\top \mathbf{x} + b,\; \sigma^2\big)}$$

A Probabilistic Model for Regression

Assumption: observations are noisy versions of a linear function.

$$y = f_\theta(\mathbf{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

With $f_\theta(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$, this means:

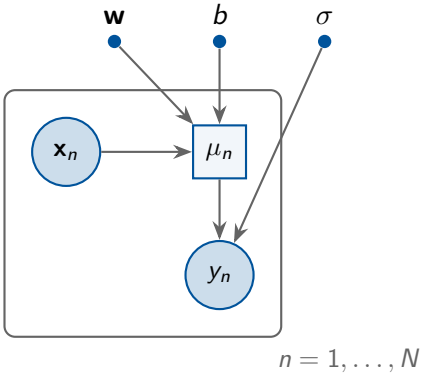
$$\boxed{y \mid \mathbf{x} \;\sim\; \mathcal{N}\big(\mathbf{w}^\top \mathbf{x} + b,\; \sigma^2\big)}$$

The model explicitly represents:

- the **signal**: $\mathbf{w}^\top \mathbf{x} + b$ (what we predict)
- the **noise**: σ^2 (how much variability remains)

Graphical Model (Plate Notation)

The probabilistic model can be represented as a **Bayesian network** using plate notation:



● = parameter / fixed value (`pyro.param`)

○ = observed r.v. (`pyro.sample`, `obs=y`)

▢ = plate (`pyro.plate`)

□ = deterministic function

$$\mu_n = \mathbf{w}^\top \mathbf{x}_n + b \qquad y_n \sim \mathcal{N}(\mu_n, \sigma^2)$$

Likelihood

Given the model, the probability of observing the data is:

$$p(\mathcal{D} \mid \theta) \;=\; \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i, \theta) \;=\; \prod_{i=1}^{N} \mathcal{N}\big(y_i \;\big|\; \mathbf{w}^\top \mathbf{x}_i + b,\; \sigma^2\big)$$

Likelihood

Given the model, the probability of observing the data is:

$$p(\mathcal{D} \mid \theta) \;=\; \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i, \theta) \;=\; \prod_{i=1}^{N} \mathcal{N}\big(y_i \;\big|\; \mathbf{w}^\top \mathbf{x}_i + b,\; \sigma^2\big)$$

This is the **likelihood function**: it measures how well the model with parameters $\theta = (\mathbf{w}, b, \sigma^2)$ explains the observed data.

Goal: find θ that *maximizes* the likelihood.

Maximum Likelihood $\Rightarrow$ MSE

Maximizing likelihood = minimizing **negative log-likelihood**:

$$
\begin{aligned}
-\log p(\mathcal{D} \mid \theta) &= -\sum_{i=1}^{N} \log \mathcal{N}\big(y_i \mid \mathbf{w}^\top \mathbf{x}_i + b,\ \sigma^2\big) \\
&= \frac{1}{2\sigma^2} \sum_{i=1}^{N} \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 \ + \ \frac{N}{2} \log \sigma^2 \ + \ \text{const.}
\end{aligned}
$$

For fixed σ^2, minimizing the NLL w.r.t. $\mathbf{w}, b$ is **exactly MSE**.

Maximum Likelihood $\Rightarrow$ MSE

Maximizing likelihood = minimizing **negative log-likelihood**:

$$
\begin{aligned}
-\log p(\mathcal{D} \mid \theta) &= -\sum_{i=1}^{N} \log \mathcal{N}\big(y_i \mid \mathbf{w}^\top \mathbf{x}_i + b,\ \sigma^2\big) \\
&= \frac{1}{2\sigma^2} \sum_{i=1}^{N} \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 \ + \ \frac{N}{2} \log \sigma^2 \ + \ \text{const.}
\end{aligned}
$$

Maximum Likelihood $\Rightarrow$ MSE

Maximizing likelihood = minimizing **negative log-likelihood**:

$$
\begin{aligned}
-\log p(\mathcal{D} \mid \theta) &= -\sum_{i=1}^{N} \log \mathcal{N}\big(y_i \mid \mathbf{w}^\top \mathbf{x}_i + b,\ \sigma^2\big) \\
&= \frac{1}{2\sigma^2} \sum_{i=1}^{N} \big(y_i - \mathbf{w}^\top \mathbf{x}_i - b\big)^2 \ + \ \frac{N}{2} \log \sigma^2 \ + \ \text{const.}
\end{aligned}
$$

For fixed σ^2, minimizing the NLL w.r.t. $\mathbf{w}, b$ is **exactly MSE**.

The Punchline

Key insight

MSE is **not** an arbitrary choice.
It is the **consequence** of assuming Gaussian noise.

The Punchline

Key insight

MSE is **not** an arbitrary choice.
It is the **consequence** of assuming Gaussian noise.

Different distributional assumptions $\Rightarrow$ different loss functions:

Noise distribution	**Resulting loss**
Gaussian $\mathcal{N}(0, \sigma^2)$	Mean Squared Error (MSE)
Laplace $\mathrm{Laplace}(0, b)$	Mean Absolute Error (MAE)

The loss function is a **consequence** of the model, not an ad-hoc design decision.

σ^2 Is Also a Parameter

σ^2 is not just a technical detail — it is a **model parameter**:

- It is estimated jointly with $\mathbf{w}$ and b via MLE.
- It quantifies the **noise level** in the data.
- It determines the **width of our uncertainty bands**.

σ^2 Is Also a Parameter

σ^2 is not just a technical detail — it is a **model parameter**:

- It is estimated jointly with $\mathbf{w}$ and b via MLE.
- It quantifies the **noise level** in the data.
- It determines the **width of our uncertainty bands**.

Recall the NLL includes a term $\frac{N}{2} \log \sigma^2$:

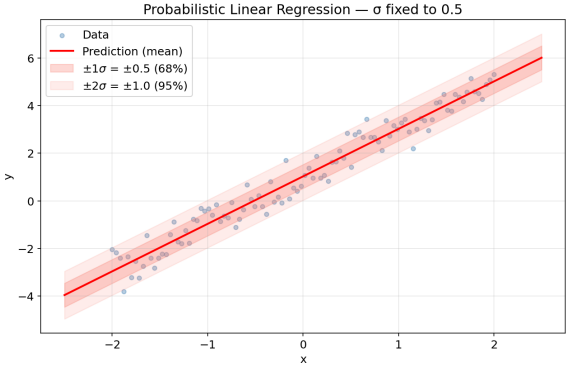
- Too small $\sigma \Rightarrow$ large penalty on residuals.
- Too large $\sigma \Rightarrow$ penalized by the log term.
- MLE finds the **right balance**.

(Extension: σ could depend on $\mathbf{x}$ — heteroscedastic noise. We won't implement this today, but Pyro supports it naturally.)

What We Gain

With a probabilistic model we obtain:

1. **Predictions** — the mean $\mathbf{w}^\top \mathbf{x} + b$.
2. **Uncertainty** — calibrated intervals via σ^2.
3. **Model comparison** — principled, via likelihood.



Preview: From Linear to Nonlinear

What if the data are **not linear**?

Replace the linear function with a **neural network**:

$$y \mid \mathbf{x} \sim \mathcal{N}\big(f_\theta(\mathbf{x}),\ \sigma^2\big) \qquad \text{where } f_\theta = \text{neural network}$$

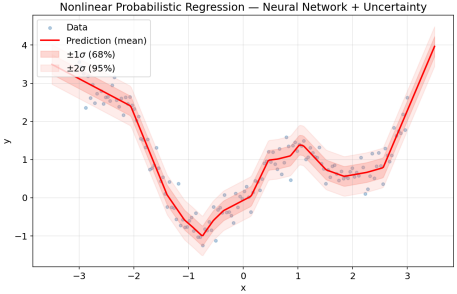
Preview: From Linear to Nonlinear

What if the data are **not linear**?

Replace the linear function with a **neural network**:

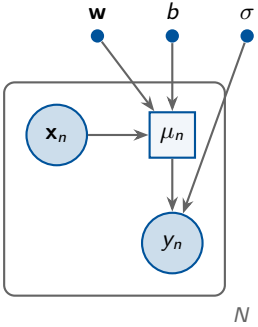
$$y \mid \mathbf{x} \sim \mathcal{N}\big(f_\theta(\mathbf{x}),\ \sigma^2\big) \qquad \text{where } f_\theta = \text{neural network}$$

The probabilistic model does not change.
Only the **function family** of the predictor changes.



Same Graphical Model, Different Function

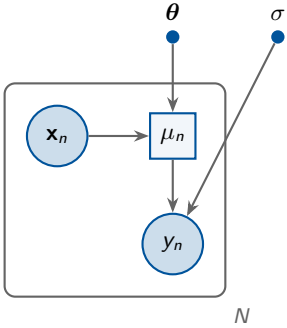
Linear regression



$$\mu_n = \mathbf{w}^\top \mathbf{x}_n + b$$

$\Rightarrow$

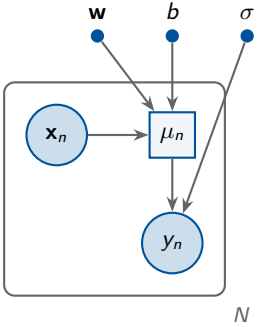
Neural network regression



$$\mu_n = f_{\boldsymbol{\theta}}(\mathbf{x}_n)$$

Same Graphical Model, Different Function

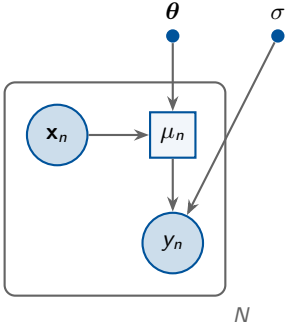
Linear regression



$$\mu_n = \mathbf{w}^\top \mathbf{x}_n + b$$

$\Rightarrow$

Neural network regression



$$\mu_n = f_{\boldsymbol{\theta}}(\mathbf{x}_n)$$

The graphical model structure is identical. Only the deterministic function $\mu_n = g(\mathbf{x}_n)$ changes — the probabilistic layer ($y_n \sim \mathcal{N}(\mu_n, \sigma^2)$ inside the plate) remains the same.

Let's See It in Code

Time to open the **Pyro notebook**!

What we will implement:

1. Define the probabilistic linear regression model in Pyro.
2. Train via maximum likelihood (SVI).
3. Visualize predictions **with uncertainty bands**.
4. Compare with the nonlinear (neural network) extension.

The model you just saw on the slides translates *directly* into Pyro code.

Our Tool: Pyro

Pyro is a probabilistic programming language built on top of **PyTorch**.

- Developed by **Uber AI Labs** (now maintained by the open-source community).
- Lets you write probabilistic models in pure Python.
- Leverages PyTorch's autograd for **automatic gradient-based inference**.
- Supports both MLE and **Bayesian inference** (Day 2).
- Scales to large datasets and deep models.

Our Tool: Pyro

Pyro is a probabilistic programming language built on top of **PyTorch**.

- Developed by **Uber AI Labs** (now maintained by the open-source community).
- Lets you write probabilistic models in pure Python.
- Leverages PyTorch's autograd for **automatic gradient-based inference**.
- Supports both MLE and **Bayesian inference** (Day 2).
- Scales to large datasets and deep models.

Key idea: you already know PyTorch — Pyro adds a thin probabilistic layer on top.

https://pyro.ai